

McMaster University
Department of Electrical and Computer Engineering

COMPENG 4DM4

Lab 1 Report

Performance in Computer Architecture

Group: 06

Submission date: Sep 31, 2026

Declaration of Contribution

Complete the entries below with each member's actual contribution to the lab.

Team member	Student ID	Contribution
Barry Bian	Bianc1	Finished Exercise 1 & part of Exercise 2 Part 1
Jack Xu	Xu503	Finished Exercises 2 Part 4 and some of part 3
Bill Zhou	Zhouz247	Finished Exercise 1 & part of Exercise 2 Part 1 & 2
Victor Trepanier	trepanv	Finished Exercises 2 Part 3

Exercise 1

Part 1 – Instruction Count PinTool

We ran Exercise 1 Part 1 in the course-provided x86-64 Linux VM, emulated with UTM. We set LAB_ROOT to the actual repository location, configured PIN_HOME and PIN_ROOT for Pin 3.13, and created the output and temporary directories. From the Pin directory, we ran source compile.sh to compile the 13 supplied benchmarks into their respective BM binaries. We then used the provided inscount0.so Pintool to collect each executable's dynamic instruction count.

Our Bash script iterated over tools/benchmarks/*/BM, ran each binary with Pin and inscount0.so, and extracted the numeric value following Count from its output file. It wrote the header benchmark_name;instruction_count and one semicolon-separated row per benchmark to report/lab1-exercise1-inscount.csv. During debugging, correcting LABOUT to LAB_OUT restored the intended output path, and replacing print with printf fixed CSV row generation. Finally, we checked the CSV with cat and wc -l. Figure 1 shows 14 lines: one header and 13 benchmark records, all containing positive integer counts.

Table 1. Dynamic instruction counts collected with inscount0.

Benchmark	Instruction count
median	158,647
memBW	31,621,301
memcpy	239,164
memlatency	22,901,585
mergesort	400,648
multiply	167,365
qsort	484,639
rsort	677,068
spmv	238,526
st-matmul	241,858
st-vvadd	248,039
towers	147,118
vvadd	145,798

memBW has the highest count (31,621,301), while vvadd has the lowest (145,798). These are counts of dynamically executed instructions; they do not by themselves measure execution time or CPI. The mergesort count (400,648) is the Pin reference for Exercise 2 Part I.

```
gp06@4dm4-virtual-machine: ~/Desktop/macsim-4dm4/4DM4...
_trace_generator/pin-3.13-98189-g60a6ef199-gcc-linux$ cat "$LAB_OUT/lab1-exercis
e1-inscount.csv"
benchmark_name;instruction_count
median;158647
memBW;31621301
memcpy;239164
memlatency;22901585
mergesort;400648
multiply;167365
qsort;484639
rsort;677068
spmv;238526
st-matmul;241858
st-vvadd;248039
towers;147118
vvadd;145798
gp06@4dm4-virtual-machine:~/Desktop/macsim-4dm4/4DM4_Lab1Template-main/tools/x86
_trace_generator/pin-3.13-98189-g60a6ef199-gcc-linux$ wc -l "$LAB_OUT/lab1-exerc
ise1-inscount.csv"
14 /home/gp06/Desktop/macsim-4dm4/4DM4_Lab1Template-main/labs/lab1/lab1-exercise
1-inscount.csv
gp06@4dm4-virtual-machine:~/Desktop/macsim-4dm4/4DM4_Lab1Template-main/tools/x86
_trace_generator/pin-3.13-98189-g60a6ef199-gcc-linux$
```

Figure 1. Exercise 1 Part 1 output and the 14-line CSV check.

Evidence supplied from the lab VM. Table 1 transcribes the counts shown above.

Part 2 – MacSim Trace Generator

We first ran the compiled mergesort benchmark using Pin 3.13 and **trace_generator.so**, specifying the trace prefix with **-tracename** and enabling readable output with **-dump** and **-dump_file**. The generated configuration contained **x86 1.3, 1**, and **0 0** (Figure 2), identifying the x86 trace format version, one traced thread, and thread 0 starting at instruction count 0. We inspected the dump's opcode, register, control-flow, and memory-access fields (Figures 3–4). For example, the **PUSH** record listed **rbp** and **rsp** as source registers, with **has_st = 1** and **mem_write_size = 8**, consistent with an eight-byte stack write. We then ran the supplied **createTraces.sh** without modification for all 13 benchmarks. A separate verification script reported OK for both Config and Raw for every benchmark, confirming a non-empty configuration file and at least one non-empty raw trace file in each benchmark directory (Figure 5). We also inspected every generated **BM.txt** and found the same contents as the standalone mergesort configuration. This is expected because all recorded configurations specify the same x86 trace format, a single thread, and the same starting instruction count. These configuration files describe how the traces are organized, while each benchmark's

instruction sequence is stored in its corresponding raw trace file. Therefore, identical configuration contents do not imply identical execution traces or instruction counts.

```
gp06@4dm4-virtual-machine:~/Desktop/macsim-4dm4/4DM4_Lab1Template-main/tools/x86_trace_generator/pin-3.13-98189-g60a6ef199-gcc-linux$ cat "$TRACE_CHECK/mergesort.t.txt"
x86 1.3
1
0 0
```

Figure 2. Standalone mergesort trace configuration showing x86 format version 1.3, one traced thread, and thread 0 starting at instruction count 0.

```
gp06@4dm4-virtual-machine: ~/Desktop/macsim-4dm4/4DM4...
gp06@4dm4-virtual-machine:~/Desktop/macsim-4dm4/4DM4_Lab1Template-main/tools/x86_trace_generator/pin-3.13-98189-g60a6ef199-gcc-linux$ head -n 40 "$TRACE_CHECK/mergesort_0.dump"
*** begin of the data structure ***
t_info->uop_opcode_num 99
t_info->uop_opcode TR_NOP
t_info->num_read_regs: 2
t_info->num_dest_regs: 0
t_info->src0: 3 : rdi
t_info->src1: 8 : rdx
t_info->cf_type: NOT_CF
t_info->has_immediate: 0
t_info->r_dir: 0
t_info->has_st: 0
t_info->num_ld: 0
t_info->mem_read_size: 0
t_info->mem_write_size: 0
t_info->is_fp: 0
t_info->ld_vaddr1: 0
t_info->ld_vaddr2: 0
t_info->st_vaddr: 0
t_info->instruction_addr: 7b99e0035030
t_info->branch_target: 0
t_info->actually taken: 0
```

Figure 3. Excerpt from mergesort_0.dump showing an instruction record's opcode, register operands, control-flow classification, and memory-access fields.

```
gp06@4dm4-virtual-machine: ~/Desktop/macsim-4dm4/4DM4...
t_info->ld_vaddr2: 0
t_info->st_vaddr: 0
t_info->instruction_addr: 7b99e0035030
t_info->branch_target: 0
t_info->actually_taken: 0
t_info->write_flg: 0
*** end of the data structure ***

*** begin of the data structure ***
t_info->uop_opcode_num 59
t_info->uop_opcode PUSH
t_info->num_read_regs: 2
t_info->num_dest_regs: 0
t_info->src0: 5 : rbp
t_info->src1: 6 : rsp
t_info->cf_type: NOT_CF
t_info->has_immediate: 0
t_info->r_dir: 0
t_info->has_st: 1
t_info->num_ld: 0
t_info->mem_read_size: 0
t_info->mem_write_size: 8
t_info->is_fp: 0
t_info->ld_vaddr1: 0
```

Figure 4. Further dump output showing a PUSH record with a store enabled and an eight-byte memory write.

```
gp06@4dm4-virtual-machine: ~/Desktop/macsim-4dm4/4DM4_Lab1Template-main/tools/x86
_trace_generator/pin-3.13-98189-g60a6ef199-gcc-linux$ bash ~/Desktop/macsim-4dm4
/4DM4_Lab1Template-main/labs/lab1/exercise1_part2_check_traces.sh
Benchmark      Config      Raw
median         OK         OK
memBW          OK         OK
memcpy         OK         OK
memlatency     OK         OK
mergesort      OK         OK
multiply       OK         OK
qsort          OK         OK
rsort          OK         OK
spmv           OK         OK
st-matmul      OK         OK
st-vvadd       OK         OK
towers         OK         OK
vvadd          OK         OK

Benchmarks checked: 13
```

Figure 5. Independent file checks reporting Config and Raw as OK for all 13 benchmarks, confirming the presence of non-empty configuration and raw trace files.

Exercise 2

Part I – First MacSim Experiment

MacSim was configured as a single core x86 system, and the trace file list was set to the mergesort benchmark. The simulation completed successfully with 397424 instructions, 4795410 cycles. The Pintools instruction count for the mergesort benchmark was 400648, while MacSim reported 397424 instructions. Therefore, the two values do not match exactly. This difference occurs because Pin counts instructions directly from the original benchmark execution, while MacSim simulates the generated instruction trace and performs its own trace processing.

```
gp06@4dm4-virtual-machine:~/Desktop/Lab1/Lab1Template-Group6/bin$ ./macsim
Arguments: ./macsim
src/macsim.cc:122: (I=0 C=0): initialize knobs
number of cores : 1
src/macsim.cc:537: (I=0 C=0): initialize simulation
src/macsim.cc:321: (I=0 C=0): initialize cores (1/0/0)
src/exec.cc:190: (I=0 C=0): UOP latency mapped to Sandy Bridge
src/memory.cc:1567: (I=0 C=0): Memory system(igpu_network) has been initialized.
src/network_ring.cc:94: (I=0 C=0): TOTAL_ROUTER:4 CPU:1 GPU:0 L3:1 LLC:1 MC:1
src/network_ring.cc:284: (I=0 C=0): RING topology
0 <-> 1 <-> 2 <-> 3 <->
enabling bug detector
src/macsim.cc:750: (I=0 C=0): Initializing new noc

src/macsim.cc:838: (I=0 C=0): Clock LCM : 110
src/macsim.cc:839: (I=0 C=0): CPU clock frequency : 1.15 GHz
src/macsim.cc:840: (I=0 C=0): GPU clock frequency : 1.15 GHz
src/macsim.cc:841: (I=0 C=0): LLC clock frequency : 1 GHz
src/macsim.cc:842: (I=0 C=0): NOC clock frequency : 1.15 GHz
src/macsim.cc:843: (I=0 C=0): MC clock frequency : 1.15 GHz
src/process_manager.cc:430: (I=0 C=0): setup_process:0 ../tools/benchmarks/mergesort/BM.txt current_index:0 (1)
trace:../tools/benchmarks/mergesort/BM.txt is opened
src/process_manager.cc:541: (I=0 C=0): thread_count:1
**Core 00 Thread 00 Finished: insts:397424 cycles:4795410 (47954090) seconds:40 -- 0.08 IPC
(0.08 IPC) -- N/A KHz (9.94 KHz)
src/retire.cc:563: (I=397423 C=47954090): ----- application 0 terminated (../tools/benchmarks/mergesort/BM.txt,0) at 47954090
**Core 00 Core_Total Finished: insts:397424 cycles:4795410 (47954090) seconds:40 -- 0.08 IPC
(0.08 IPC) -- N/A KHz (9.94 KHz)
src/macsim.cc:620: (I=397424 C=47954091): finalize simulation
src/macsim.cc:624: (I=397424 C=47954091): elapsed time:40.3 seconds
Done
```

Figure 6. Output of the macsim run for mergesort.

Part II – Program Analysis

debug_exec_stage was set to 1, and a script called exercise2_part2_macsim_script.sh in bin folder was used to run all 13 benchmarks automatically. The MacSim output for each benchmark was saved, and the done_exec lines were extracted to obtain the completed UOP types.

A Python script called exercise2_part2_generate_uop_csv.py in bin folder then counted each CPU UOP type and generated UOP_No.csv, with benchmarks as columns and UOP types as rows.

UOP	median	memBW	memcpy	memlatency	mergesort	multiply	qsort	rsort	spmv	st-matmul	st-vvadd	towers	vvadd
UOP_BYTE	0	3	0	2	0	0	0	0	0	0	0	0	0
UOP_CF	29013	1867576	32911	2254818	56172	33670	70825	35533	30807	31849	31559	27966	27364
UOP_CMOV	264	721284	268	664	261	264	264	268	764	262	2262	264	264
UOP_FADD	1359	2098	1353	199520	1358	1359	1357	1353	6756	1332	7330	1357	1357
UOP_FDIV	0	5	0	7	0	0	0	0	0	0	0	0	0
UOP_FMEM	1111	263793	3123	67379	1099	1111	1115	1127	19211	1102	25106	1115	1115
UOP_FMUL	0	1	0	0	0	0	0	0	2399	0	0	0	0
UOP_IADD	90559	13397684	125415	10703272	220528	92897	237317	318646	128002	131452	130311	80804	83994
UOP_IMEM	55493	14928220	88958	15846925	200453	70722	266504	339536	75131	113098	83534	52119	48077
UOP_IMUL	144	243	145	65883	142	144	144	145	144	12690	144	144	144
UOP_LOGIC	14272	738620	14390	410954	15297	20672	14260	14385	16760	14213	16203	14270	14260
UOP_NOP	1643	2293	1636	395398	5125	1740	10926	1636	1642	1638	1641	2481	1642
UOP_SHIFT	4093	3542882	7310	528240	5295	9794	9148	31990	3790	3533	5275	3307	3590
total UOPS	197951	35464702	275509	30473062	505730	232373	611860	744619	285406	311169	303365	183827	181807

Figure 7. UOP_No.csv

Part III – CPI Performance Calculations

The manual CPI was calculated using the UOP counts and their corresponding latencies. The MacSim CPI was calculated from the total cycles divided by the total instructions. All of the calculations are automatically computed by the script `exercise2_part3_generate_cpi.py` in `bin` folder. The formula is given below:

$$CPI_{manual} = \frac{\sum UOP_{count} \times UOP_{latency}}{\sum UOP_{count}}, \quad CPI_{macsim} = \frac{Total\ Cycles}{Total\ Instructions}$$

The manually calculated CPI values are all close to 10, while the MacSim CPI values vary more significantly between benchmarks. Several benchmarks, such as `median`, `spmv`, `stmatmul`, `stvvadd`, `towers`, and `vvadd`, showed CPI values relatively close to the manually calculated results. However, benchmarks such as `memBW`, `memlatency`, `mergesort`, `multiply`, and especially `qsort` showed noticeably higher CPI values in MacSim. This indicates that the simple manual CPI calculation provides a reasonable baseline, but the actual performance of some benchmarks is strongly affected by additional execution and memory-system behavior modeled by MacSim.

UOP	median	memBW	memcpy	memlatency	mergesort	multiply	qsort	rsort	spmv	stmatmul	stvvadd	towers	vvadd
UOP_BYTE	0	3	0	2	0	0	0	0	0	0	0	0	0
UOP_CF	29013	1867576	32911	2254818	56172	33670	70825	35533	30807	31849	31559	27966	27364
UOP_CMOV	264	721284	268	664	261	264	264	268	764	262	2262	264	264
UOP_FADD	1359	2098	1353	199520	1358	1359	1357	1353	6756	1332	7330	1357	1357
UOP_FDIV	0	5	0	7	0	0	0	0	0	0	0	0	0
UOP_FMEM	1111	263793	3123	67379	1099	1111	1115	1127	19211	1102	25106	1115	1115
UOP_FMUL	0	1	0	0	0	0	0	0	2399	0	0	0	0
UOP_IADD	90559	13397684	125415	10703272	220528	92897	237317	318646	128002	131452	130311	80804	83994
UOP_IMEM	55493	14928220	88958	15846925	200453	70722	266504	339536	75131	113098	83534	52119	48077
UOP_IMUL	144	243	145	65883	142	144	144	145	144	12690	144	144	144
UOP_LOGIC	14272	738620	14390	410954	15297	20672	14260	14385	16760	14213	16203	14270	14260
UOP_NOP	1643	2293	1636	395398	5125	1740	10926	1636	1642	1638	1641	2481	1642
UOP_SHIFT	4093	3542882	7310	528240	5295	9794	9148	31990	3790	3533	5275	3307	3590
total UOPS	197951	35464702	275509	30473062	505730	232373	611860	744619	285406	311169	303365	183827	181807
CPI	9.9451	9.9995	9.9607	9.9476	9.9785	9.9532	9.9823	9.9855	9.7434	9.9658	9.8067	9.9409	9.9403
Total_Instructions	155535	27881896	222329	22895978	397424	164254	481499	673722	235386	238553	244686	143959	142659
Total_cycles	1640587	475418006	1972200	368605914	4795410	2318468	10565156	6070442	2207473	2467348	2436113	1475000	1395382
CPI_macsim	10.548	17.0511	8.8706	16.0992	12.0662	14.1151	21.9422	9.0103	9.3781	10.343	9.9561	10.246	9.7812

Figure 8. avg_CPI.csv

Part IV – Performance Improvement

We selected the most common UOP for each benchmark from Part II. We changed only that UOP type's latency. And then we kept all the other latencies the same. We edited `def/uoplatency_x86.def` and recompiled MacSim in debug mode.

UOP_IMEM was selected as the most frequent UOP for memBW, memlatency, qsort, and rsort, while UOP_IADD was selected for the remaining benchmarks.

For Group 6, the required latency reduction factor is: $\frac{(6+55) \times 10}{110} \approx 5.545$

For both groups, the latency was reduced from 10 cycles to 2 cycles: $\frac{10}{5.545} = 1.803 \approx 2$

MacSim was rebuilt, and the benchmarks were executed using the `exercise2_part4_change_UOP_IMEM.sh` script and `exercise2_part4_change_UOP_IADD.sh` script separately.

The results from both groups were combined and used to recalculate the manual CPI, total cycles, and MacSim CPI for the final `avg_CPI_improved.csv` file.

Compared with the original results without latency reduction, the reduced-latency configuration improved the MacSim CPI for most benchmarks. The largest improvements were observed for `vvadd` (8.13%), `median` (8.03%), and `towers` (8.02%), while `multiply`, `spmv`, `stvvadd`, `memcpy`, `stmatmul`, and `mergesort` also showed smaller CPI reductions. In contrast, `memBW`, `memlatency`, `qsort`, and `rsort` showed no change in MacSim CPI after reducing UOP_IMEM latency. Overall, reducing UOP_IADD latency produced measurable performance improvements, while reducing UOP_IMEM latency did not affect the simulated CPI for these four benchmarks.

UOP	median	memBW	memcpy	memlatency	mergesort	multiply	qsort	rsort	spmv	stmatmul	stvvadd	towers	vvadd
UOP_BYTE	0	3	0	2	0	0	0	0	0	0	0	0	0
UOP_CF	29013	1867576	32911	2254818	56172	33670	70825	35533	30807	31849	31559	27966	27364
UOP_CMOV	264	721284	268	664	261	264	264	268	764	262	2262	264	264
UOP_FADD	1359	2098	1353	199520	1358	1359	1357	1353	6756	1332	7330	1357	1357
UOP_FDIV	0	5	0	7	0	0	0	0	0	0	0	0	0
UOP_FMEM	1111	263793	3123	67379	1099	1111	1115	1127	19211	1102	25106	1115	1115
UOP_FMUL	0	1	0	0	0	0	0	0	2399	0	0	0	0
UOP_IADD	90559	13397684	125415	10703272	220528	92897	237317	318646	128002	131452	130311	80804	83994
UOP_IMEM	55493	14928220	88958	15846925	200453	70722	266504	339536	75131	113098	83534	52119	48077
UOP_IMUL	144	243	145	65883	142	144	144	145	144	12690	144	144	144
UOP_LOGIC	14272	738620	14390	410954	15297	20672	14260	14385	16760	14213	16203	14270	14260
UOP_NOP	1643	2293	1636	395398	5125	1740	10926	1636	1642	1638	1641	2481	1642
UOP_SHIFT	4093	3542882	7310	528240	5295	9794	9148	31990	3790	3533	5275	3307	3590
total UOPS	197951	35464702	275509	30473062	505730	232373	611860	744619	285406	311169	303365	183827	181807
CPI	6.2852	6.6321	6.319	5.7874	6.49	6.755	6.4977	6.3376	6.1555	6.5862	6.3703	6.4244	6.2443
Total_Instructions	155535	27881896	222329	22895978	397424	164254	481499	673722	235386	238553	244686	143959	142659
Total_cycles	1508895	475418006	1869855	368605914	4568458	2169621	10565156	6070442	2068034	2344433	2305676	1356672	1281866
CPI_macsim	9.7013	17.0511	8.4103	16.0992	11.4952	13.2089	21.9422	9.0103	8.7857	9.8277	9.423	9.424	8.9855

Program	Old CPI_macsim	New CPI_macsim	CPI Improved Percentage
median	10.548	9.7013	8.03%
memBW	17.0511	17.0511	0.00%
memcpy	8.8706	8.4103	5.19%
memlatency	16.0992	16.0992	0.00%
mergesort	12.0662	11.4952	4.73%
multiply	14.1151	13.2089	6.42%
qsort	21.9422	21.9422	0.00%
rsort	9.0103	9.0103	0.00%
spmv	9.3781	8.7857	6.32%
stmatmul	10.343	9.8277	4.98%
stvvadd	9.9561	9.423	5.35%
towers	10.246	9.424	8.02%
vvadd	9.7812	8.9855	8.13%